Workshop III Practical — Capabilities, Type-State, Abilities & Tests

In this workshop, you'll harden ScholarFlow with multi-role capabilities, lifecycle/state-machine guards, and Move unit tests.

Quick recap

- You published a package and implemented a simple mint flow.
- You created a shared `Registry` and indexed mints with a PTB.

What's next

Make real contracts safe by design. Design multi-role access control, encode lifecycle transitions as explicit entry functions, use abilities to prevent footguns, and lock your rules in with unit tests.

Learning outcomes

- Implement role capabilities (admin/issuer/revoker) and safe rotation.
- Enforce type-state lifecycle: Pending → Active → Revoked, with illegal transitions blocked.
- Audit and fix abilities (key | store | copy | drop) to match intent and conservation.
- Write Move unit tests for authorization, transitions, idempotence, and error codes.

Prerequisites

- From I & II: a working package with `grant` and (optionally) `registry`.
- IOTA CLI installed.
- Focus here is pure Move + tests.

Export convenience variables (optional on-chain trials):

```
export PKG_ID=<0x...package id>
export ADMIN_CAP_ID=<0x...admin cap object id>
export PUBLISHER_ADDR=<0x...publisher address>
```

Let's plan the hardening

- Replace “single admin does everything” with role caps.
- Make lifecycle explicit: request → approve → revoke (no raw status without guards).
- Tighten abilities on caps and resources (no accidental copy/drop).
- Prove rules with unit tests (positive and expected-failure paths).

Access control (roles)

Create `sources/access.move`. Define roles, rotation, and checks.

```
module scholarflow::access {
    use iota::object::{Self, UID};
    use iota::tx_context::{Self, TxContext};
    use iota::transfer;
    use iota::event;
    use iota::table::{Self as table, Table};
    use scholarflow::grant; // Reuse the AdminCap minted in
grant::init

    /// Errors
    const ENotAdmin: u64 = 1;
    const ENotIssuer: u64 = 2;
    const ENotRevoker: u64 = 3;
    const EAlreadyHasRole: u64 = 4;
    const ENoSuchRole: u64 = 5;

    // Capability is defined in scholarflow::grant::AdminCap (minted
in init).

    /// Roles tracked in a shared Roles object.
    public struct Roles has key {
        id: UID,
        admins: Table<address, bool>,
        issuers: Table<address, bool>,
        revokers: Table<address, bool>,
    }

    /// Events for auditability.
    public struct RoleGranted has copy, drop, store { role:
```

```
vector<u8>, who: address, by: address }
    public struct RoleRevoked has copy, drop, store { role:
vector<u8>, who: address, by: address }

    /// Create and share role registry (seeded with publisher as
admin).
    public entry fun create_roles(_cap: &grant::AdminCap, ctx: &mut
TxContext) {
        // Step 1: Get the transaction sender address.
        // Step 2: Create a new Roles object with empty
admin/issuer/revoker tables.
        // Step 3: Add the sender address to the admins table.
        // Step 4: Emit a RoleGranted event noting role "admin", who =
sender, by = sender.
        // Step 5: Share the Roles object so it becomes a shared
on-chain object.
        abort 9001
    }

    /// Grant a role (admin-gated).
    public entry fun grant_role(
        _roles: &mut Roles, _role: vector<u8>, _who: address, _cap:
&grant::AdminCap
    ) {
        // Gate with AdminCap (_cap). Suggested flow:
        // 1) Match role bytes: b"admin" | b"issuer" | b"revoker".
        // 2) Insert _who -> true into the corresponding table
(admins/issuers/revokers).
        // 3) event::emit(RoleGranted { role: _role, who: _who, by: /*
optional: actor */ 0x0 });
        //    If you want to record the actor, add a TxContext param
and set by = tx_context::sender(ctx).
        abort 9001
    }

    /// Revoke a role (admin-gated).
    public entry fun revoke_role(
        _roles: &mut Roles, _role: vector<u8>, _who: address, _cap:
&grant::AdminCap
    ) {
        // Gate with AdminCap (_cap). Suggested flow:
        // 1) Match role bytes and check presence with
table::contains; abort with ENoSuchRole if missing.
        // 2) Remove _who from the matching table via table::remove.
        // 3) event::emit(RoleRevoked { role: _role, who: _who, by: /*
optional actor */ 0x0 });
        abort 9001
    }

    /// Read helpers (pure) — used by lifecycle checks.
```

```
    public fun is_admin(_roles: &Roles, _addr: address): bool {
        // Step 1: Check if `_addr` exists in `roles.admins`.
        // Step 2: Return true if present, false otherwise.
        abort 9001
    }
    public fun is_issuer(_roles: &Roles, _addr: address): bool {
        // Step 1: Check if `_addr` exists in `roles.issuers`.
        // Step 2: Return true if present, false otherwise.
        abort 9001
    }
    public fun is_revoker(_roles: &Roles, _addr: address): bool {
        // Step 1: Check if `_addr` exists in `roles.revokers`.
        // Step 2: Return true if present, false otherwise.
        abort 9001
    }

    // test helpers removed by design; see tests module for inline
setup.
}
```

> BIG PICTURE
>
> Roles let you separate duties (admin vs issuer vs revoker) and rotate safely without republishing contracts.

Lifecycle (pending queue + approval mint)

Create `sources/lifecycle.move`. Encode a pending queue in a shared object and mint on approval.

```
module scholarflow::lifecycle {
    use iota::event;
    use iota::object::{Self as object, UID, ID};
    use iota::transfer;
    use iota::tx_context::{Self as tx_context, TxContext};
    use iota::table::{Self as table, Table};
    use scholarflow::{access, grant};
    use std::vector;

    const EOnlyIssuer: u64 = 10;
    const EOnlyRevoker: u64 = 11;
    const EInvalidTransition: u64 = 12;
    const ENoRequest: u64 = 13;

    const S_PENDING: vector<u8> = b"pending";
    const S_ACTIVE: vector<u8>  = b"active";
```

```
    const S_REVOKED: vector<u8> = b"revoked";

    /// Emitted when a student places/updates a grant request with an
amount.
    public struct GrantRequested has copy, drop, store { student:
address, amount: u64 }
    public struct GrantApproved  has copy, drop, store { grant_id: ID,
student: address }
    public struct GrantRevoked   has copy, drop, store { grant_id: ID,
by: address }

    /// Shared queue of requested grants: student -> requested amount.
    public struct Requests has key {
        id: UID,
        by_student: Table<address, u64>,
    }

    /// Create and share the Requests queue.
    public entry fun create_requests(_cap: &grant::AdminCap, _ctx:
&mut TxContext) {
        // Step 1: Create Requests { id: object::new(ctx), by_student:
table::new<address,u64>(ctx) }.
        // Step 2: share_object the queue.
        abort 9001
    }

    /// A student requests a grant; upsert (student -> amount) and
emit event.
    public entry fun request_grant(
        _requests: &mut Requests,
        _amount: u64,
        _ctx: &mut TxContext,
    ) {
        // Step 1: student = tx_context::sender(ctx).
        // Step 2: If contains, remove existing; then insert student -
> amount.
        // Step 3: event::emit(GrantRequested { student, amount }).
        abort 9001
    }

    /// Approve a student's pending request: remove from queue, mint
via grant, transfer to student, emit event.
    public entry fun approve_grant(
        _requests: &mut Requests,
        _student: address,
        _roles: &access::Roles,
        _cap: &grant::AdminCap,
        _ctx: &mut TxContext
    ) {
        // Step 1: assert!(access::is_issuer(roles, sender)).
```

```
        // Step 2: assert pending exists for student, read & remove
amount.
        // Step 3: call grant::mint_return_id(student, amount,
/*state=*/ S_ACTIVE, cap, ctx).
        // Step 4: emit GrantApproved with the returned ID.
        abort 9001
    }

    /// Mark ACTIVE grant as REVOKED and emit event (issuer/ or
revoker-gated per policy).
    public entry fun revoke_grant(_g: &mut grant::Grant, _roles:
&access::Roles, _ctx: &mut TxContext) {
        // Step 1: Gate with your chosen role (issuer or revoker).
        // Step 2: Ensure state is ACTIVE (if your grant::Grant
encodes lifecycle state).
        // Step 3: Set state to REVOKED via grant helper if provided,
or only emit event.
        // Step 4: event::emit(GrantRevoked { grant_id: object::id(g),
by: sender }).
        abort 9001
    }

    /// Optional helpers (if you expose state helpers from grant
module).
    // public fun is_active(_g: &grant::Grant): bool { abort 9001 }
    // public fun state_of(_g: &grant::Grant): vector<u8> { abort 9001
}
}
```

> **DESIGN CHOICE**
>
> We record "pending" requests on-chain in a shared queue and mint on approval to avoid multi-signer flows. Keep transitions centralized and avoid cross-module state mutation unless your grant module exposes explicit helpers.

Grant changes (helpers + mint API)

Update `sources/grant.move` to expose lifecycle-friendly helpers and a mint that accepts a state tag.

```
module scholarflow::grant {
    use iota::object::{Self as object, UID, ID};
    use iota::tx_context::{Self as tx_context, TxContext};
    use iota::transfer;
    use iota::event;
```

```
    use std::vector;

    /// Capability granting authority to mint grants.
    public struct AdminCap has key { id: UID }

    /// An owned grant object assigned to a student with a lifecycle
state.
    public struct Grant has key {
        id: UID,
        student: address,
        amount: u64,
        state: vector<u8>,
    }

    /// Emitted when a grant is minted.
    public struct GrantMinted has copy, drop, store { student:
address, amount: u64, grant_id: ID }

    /// Runs once at package publish. Transfers AdminCap to the
publisher.
    fun init(_ctx: &mut TxContext) { /* seed AdminCap to publisher */
}

    /// Construct a grant (module-internal helper).
    public fun create(_student: address, _amount: u64, _state:
vector<u8>, _ctx: &mut TxContext): Grant {
        // Step: return a new Grant { id: object::new(ctx), student,
amount, state }
        abort 9001
    }

    /// Read/mutate helpers for other modules (optional if you keep
state external).
    public fun get_state(_g: &Grant): vector<u8> { /* return a copy */
abort 9001 }
    public fun set_state(_g: &mut Grant, _state: vector<u8>) { /*
assign */ abort 9001 }
    public fun state_eq(_g: &Grant, _tag: &vector<u8>): bool { /*
byte-wise compare */ abort 9001 }

    /// Mint to a student with a chosen lifecycle state and emit
event.
    public entry fun mint(
        _student: address,
        _amount: u64,
        _state: vector<u8>,
        _cap: &AdminCap,
        _ctx: &mut TxContext
    ) { /* create -> emit -> transfer */ abort 9001 }
```

```
    /// Mint and return the new Grant ID (for PTB composition /
callers like lifecycle::approve_grant).
    public entry fun mint_return_id(
        _student: address,
        _amount: u64,
        _state: vector<u8>,
        _cap: &AdminCap,
        _ctx: &mut TxContext
    ): ID { /* create -> emit -> transfer -> return id */ abort 9001 }
}
```

Abilities audit (guided)

Tighten abilities to match intent and conservation rules.

- `AdminCap` and role tables should not be copyable; prefer only `key | store` where necessary. |- `Grant` should not be copyable or silently droppable. Provide explicit burn/delete entry if destruction is allowed.
- If delete is allowed, decide who can delete and how indexes (e.g., `registry`) must be updated.
- Add asserts that enforce conservation (no duplicates; explicit ownership transfers).

Tests: lock in rules

Create `tests/tests_lifecycle.move`. Write positive and expected-failure tests with named abort codes.

```
module scholarflow::tests_lifecycle {
    use 0x1::test; // adjust to your stdlib path
    use scholarflow::access;
    use scholarflow::lifecycle;
    use iota::test_scenario::{Self as ts, Scenario};

    const ADMIN: address = @0xAAA1;
    const ISSUER: address = @0xAAA2;
    const REVOKER: address = @0xAAA3;
    const STUDENT: address = @0xBBB1;

    /// Arrange helpers: seed roles inline for tests.
    fun setup(_ctx: &mut ts::Scenario): access::Roles {
        // Step 1: Create a Roles object with fresh UID and empty
tables for admins/issuers/revokers.
        // Step 2: Insert ADMIN into admins, ISSUER into issuers,
REVOKER into revokers.
```

```
        // Step 3: Return the configured Roles value for use in tests.
        abort 9001
    }

    #[test]
    #[expected_failure(abort_code = lifecycle::EOnlyIssuer)]
    fun request_requires_issuer() {
        // Steps:
        // 1) Begin scenario with STUDENT (non-issuer).
        // 2) Call setup to create roles mapping without STUDENT as
issuer.
        // 3) Attempt lifecycle::request_grant; expect abort
EOnlyIssuer.
    }

    #[test]
    fun request_succeeds_for_issuer() {
        // Steps:
        // 1) Begin scenario with ISSUER.
        // 2) Call setup to include ISSUER in issuers table.
        // 3) Call lifecycle::request_grant; assert no abort
(optionally verify state is PENDING).
    }

    #[test]
    #[expected_failure(abort_code = lifecycle::EOnlyIssuer)]
    fun approve_requires_issuer() {
        // Suggested approach:
        // 1) Begin scenario with ISSUER; setup roles; request_grant
to create a pending grant.
        // 2) Next tx with STUDENT (not issuer); take the Grant owned
by ISSUER or park it for borrowing.
        // 3) Call lifecycle::approve_grant(&mut grant, &roles) as
non-issuer; expect abort EOnlyIssuer.
    }

    #[test]
    #[expected_failure(abort_code = lifecycle::EOnlyRevoker)]
    fun revoke_requires_revoker() {
        // Suggested approach:
        // 1) Begin scenario as ISSUER; create a pending grant, then
approve it (ACTIVE) with an issuer.
        // 2) Next tx as STUDENT (non-revoker); call revoke_grant on
the ACTIVE grant; expect EOnlyRevoker.
    }

    #[test]
    fun valid_transition_pending_to_active() {
        // Suggested approach:
        // 1) Begin scenario as ISSUER; setup roles; request_grant to
```

```
create PENDING grant.
        // 2) Approve as ISSUER; assert lifecycle::is_active(&grant)
== true.
        //    You can retrieve/return objects with
test_scenario::take_from_sender / return_to_sender.
    }

    #[test]
    #[expected_failure(abort_code = lifecycle::EInvalidTransition)]
    fun invalid_transition_active_to_pending() {
        // Suggested approach:
        // 1) Move grant to ACTIVE via approve.
        // 2) Attempt to transition back to PENDING (e.g., by calling
request/approve logic incorrectly) and expect EInvalidTransition.
    }

    #[test]
    fun idempotent_indexing_example_if_applicable() {
        // If you maintain any indexes (e.g., registry), upsert twice
with same values
        // and assert there is no duplication or inconsistent state.
    }
}
```

Build & run tests

```
iota move build
iota move test
```

(Optional) On-chain smoke checks

If you want to try entries against a devnet/testnet node, wire minimal calls (no PTB/SDK yet):

```
iota client call \
  --package $PKG_ID \
  --module access \
  --function create_roles \
  --args $ADMIN_CAP_ID \
  --gas-budget 60000000
```

Verification checklist

- Only admins can grant/revoke roles; issuer/revoker gates work.
- Pending → Active → Revoked enforced; illegal paths abort with named codes.
- Abilities align with intent (no accidental copy/drop on caps/resources).
- Tests cover positive/negative paths and any idempotent writes.

Common pitfalls

> ⚠ **CAUTION**
>
> - Over-permissive abilities on caps or grants → remove copy/drop.
> - Single "status" without guards → enforce transitions in entries; never mutate `state` directly.
> - No negative tests → add expected-failure tests for each guard.
> - Forgot index cleanup on delete → specify update order and assert post-conditions in tests.

Wrap-up & next (Workshop IV)

You now have principled access control, lifecycle safety, and tests. Next, we'll integrate with the Rust SDK: build PTBs, read objects and events, and maintain a tiny projection service — using the guarantees you just encoded.

Give Feedback

Tags: Tutorial | Move Smart Contracts | Testing

Previous
« Core II: Coordination with Shared Objects, Dynamic Fields, and PTBs

Next
Core IV: Wrapping it up - Dapp »